

```

1  (*
2                                CS51 Lab 5
3                                Variants, algebraic types, and pattern matching
4  *)
5
6  (*
7                                SOLUTION
8  *)
9
10 (*=====
11 Part 1: Colors as an algebraic data type
12
13 In this lab you'll use algebraic data types to create several data
14 structures.
15
16 Ultimately, you'll define several types and structures that allow you to
17 create a family tree. To do this, you need to create a type to store a
18 set of biographical information about a person, like name, birthdate,
19 and favorite color. This set of data is different from the enrollment
20 data from the prior lab, so you'll need to create a new type.
21
22 You might be tempted to do something simple like
23
24     type person = { name : string; favorite : string; birthday : string } ;;
25
26 Let's consider why this may not be appropriate by evaluating the type
27 for each record field individually.
28
29 It seems reasonable for a name to be a string (though personal name
30 structures can be quite complex; see
31 <https://en.wikipedia.org/wiki/Personal\_name>), so let's declare that
32 complete and move on.
33
34 The "favorite" field is more problematic. Although we named it such for
35 simplicity, it doesn't convey very well that we intended for this field
36 to represent a person's favorite *color*. This could be resolved with
37 some documentation, but is not enforced at any level other than hope.
38 Further, it's very likely that many people would select one of a subset
39 of simple colors. Let's fix this issue first.
40
41 .....
42 Exercise 1: Define a new algebraic type, called `color_label`, whose
43 values can be any of red, orange, yellow, green, blue, indigo, or
44 violet, each specified by an appropriate constructor.
45 .....*)
46
47 type color_label =
48     | Red
49     | Orange
50     | Yellow
51     | Green
52     | Blue

```

```

53 | Indigo
54 | Violet ;;
55
56 (* This is a good start, but doesn't allow for definition of *all* of
57 the colors that, say, a computer display might be able to present. Let's
58 make it more expressive.
59
60 One of the most commonly used methods of representing color in digital
61 devices is as an "RGB" value: a triplet of values to represent red,
62 green, and blue components that, through additive mixing, produce the
63 wide array of colors our devices render.
64
65 Commonly, each of the red, green, and blue values are made up of a
66 single 8-bit (1-byte) integer. Since one byte represents  $2^{*8} = 256$ 
67 discrete values, there are over 16.7 million ( $256 * 256 * 256$ )
68 possible colors that can be represented with this method.
69
70 The three components that make up an RGB color are referred to as
71 "channels". In this 8-bit-per-channel model, a value of 0 represents no
72 color and a value of 255 represents the full intensity of that color.
73 Some examples:
74
75   R | G | B | Color
76   ---|---|---|-----
77   255| 0 | 0 | Red
78   164| 16| 52| Crimson
79   255| 165| 0 | Orange
80   255| 255| 0 | Yellow
81   0 | 64 | 0 | Dark green
82   0 | 255| 0 | Green
83   0 | 255| 255| Cyan
84   0 | 0 | 255| Blue
85   75 | 0 | 130| Indigo
86   240| 130| 240| Violet
87
88 .....
89 Exercise 2: Define an algebraic color type that supports either
90 `Simple` colors (from the `color_label` type you defined previously)
91 or `RGB` colors, which would incorporate a tuple of values for the
92 three color channels. You'll want to use `Simple` and `RGB` as the
93 value constructors in this new variant type.
94 .....*)
95
96 type color =
97   | Simple of color_label
98   | RGB of int * int * int ;;
99
100 (* There is an important assumption about the RGB values that determine
101 whether a color is valid or not. The RGB type presupposes an
102 *invariant*, that is, a condition that we assume to be true in order for
103 the type to be valid:
104
105 INVARIANT: The red, green, and blue channels must each be a
106 non-negative 8-bit int. Therefore, each channel must be in the range

```

```

107     [0, 255].
108
109     Since OCaml, unlike some other languages, does not have native support
110     for unsigned 8-bit integers, you should ensure the invariant remains
111     true in your code. (You might think to use the OCaml `char` type --
112     which is an 8-bit character -- but this would be an abuse of the type.
113     In any case, thinking about invariants will be useful practice for
114     upcoming problem sets.)
115
116     We'll want a function to validate the invariant for RGB color values.
117     (Simple colors are assumed always valid.) There are several approaches
118     to building such functions, which differ in their types, and for which
119     we'll use different naming conventions:
120
121     * valid_rgb : color -> bool
122
123       Returns true if the color argument is valid, false otherwise.
124
125     * validated_rgb : color -> color
126
127       Returns its argument unchanged if it is a valid color, and raises an
128       appropriate exception otherwise.
129
130     * validate_rgb : color -> unit
131
132       Returns unit; raises an appropriate exception if its argument is not
133       a valid color.
134
135     The name prefixes "valid_", "validated_", and "validate_" are intended
136     to be indicative of the different approaches to validation.
137
138     In this lab, we'll use the "validated_" approach and naming
139     convention, though you may want to think about the alternatives. (In
140     the next lab, we use the "valid_" alternative approach.)
141
142     .....
143     Exercise 3: Write a function `validated_rgb` that accepts a `color` and
144     returns that color unchanged if it's valid. However, if its argument is
145     not a valid color (that is, the invariant is violated), it raises an
146     `Invalid_color` exception (defined below) with a useful message.
147
148     For instance:
149
150       # validated_rgb (RGB (10, 20, 30)) ;;
151       - : color = RGB (10, 20, 30)
152       # validated_rgb (RGB (10, 20, 300)) ;;
153       Exception: Invalid_color "bad blue channel".
154       # validated_rgb (Simple Red) ;;
155       - : color = Simple Red
156
157     .....*)
158
159     exception Invalid_color of string ;;
160

```

```

161 let validated_rgb (color : color) : color =
162   let bad (channel : int) : bool =
163     channel < 0 || channel > 255 in
164   match color with
165   | Simple _label -> color
166   | RGB (r, g, b) ->
167     if bad r then raise (Invalid_color "bad red channel")
168     else if bad g then raise (Invalid_color "bad green channel")
169     else if bad b then raise (Invalid_color "bad blue channel")
170     else color ;;
171
172 (*.....*)
173 Exercise 4: Write a function `make_color` that accepts three integers
174 for the channel values and returns a value of the color type. Be sure
175 to verify the invariant.
176 .....*)
177
178 let make_color (r : int) (g : int) (b : int) : color =
179   validated_rgb (RGB (r, g, b)) ;;
180
181 (*.....*)
182 Exercise 5: Write a function `rgb_of_color` that accepts a `color` and
183 returns a 3-tuple of integers representing that color. This is trivial
184 for `RGB` colors, but not quite so easy for the hard-coded `Simple`
185 colors. Fortunately, we've already provided RGB values for simple
186 colors in the table above.
187 .....*)
188
189 let rgb_of_color (c : color) : int * int * int =
190   match c with
191   | RGB (r, g, b) -> (r, g, b)
192   | Simple x ->
193     match x with
194     | Red -> (255, 0, 0)
195     | Orange -> (255, 165, 0)
196     | Yellow -> (255, 255, 0)
197     | Green -> (0, 255, 0)
198     | Blue -> (0, 0, 255)
199     | Indigo -> (75, 0, 130)
200     | Violet -> (240, 130, 240) ;;
201
202 (*=====*)
203 Part 2: Dates as a record type
204
205 Now let's move on to the last data type that will be used in the
206 biographical data type: the date field.
207
208 Above, we naively proposed a string for the date field. Does this make
209 sense for this field? Arguably not, since it will make comparison and
210 calculation extremely difficult.
211
212 Dates are frequently needed in programming, and OCaml (like many
213 languages) supports them through a library module; the `Unix` module
214 provides a `tm` data type for dates and times. Normally, we would

```

```

215 reduce duplication of code by relying on that module (the edict of
216 irredundancy), but for the sake of practice you'll develop your
217 own simple version.
218
219 .....
220 Exercise 6: Create a type `date` that supports values for years,
221 months, and days. First, consider what types of data each value should
222 be. Then, consider the implications of representing the overall data
223 type as a tuple or a record.
224 .....*)
225
226 (* The `Unix` module provides for dates and times through a record
227 type, `tm`. There are other plausible alternatives, such as a
228 triple of type `int * int * int`, though the record type has a nice
229 self-documenting aspect in the labels. *)
230
231 type date = { year : int; month : int; day : int } ;;
232
233 (* After you've thought it through, go to <http://url.cs51.io/lab5-1> to
234 see how we implemented the `date` type. If you picked differently, why
235 did you choose that way? Why might our approach be preferable?
236
237 .....
238 Exercise 7: Change your `date` data type, above, to implement it in a
239 manner identical to our method. If no changes are required...well, that
240 was easy.
241 .....
242
243 Like the color type, above, date values obey invariants. In fact, the
244 invariants for this type are more complex: we must ensure that days
245 fall within an allowable range depending on the month, and even on the
246 year.
247
248 The invariants are as follows:
249
250 - For our purposes, we'll only support non-negative years.
251
252 - January, March, May, July, August, October, and December have 31
253 days.
254
255 - April, June, September, and November have 30 days.
256
257 - February has 28 days in common years, 29 days in leap years.
258
259 - Leap years are years that can be divided by 4, but not by 100,
260 unless by 400.
261
262 You may find Wikipedia's leap year algorithm pseudocode useful:
263 <https://en.wikipedia.org/wiki/Leap_year#Algorithm>
264
265 .....
266 Exercise 8: Create a `validated_date` function that raises
267 `Invalid_date` if the invariant is violated, and returns the date
268 unchanged if valid.

```

```

269 .....*)
270
271 exception Invalid_date of string ;;
272
273 let validated_date ({year; month; day} as date) : date =
274   if year < 0 then raise (Invalid_date "negative year")
275   else
276     let is_leap = (year mod 4 = 0 && year mod 100 <> 0)
277                 || year mod 400 = 0 in
278     let max_days =
279       match month with
280       | 1 | 3 | 5 | 7 | 8 | 10 | 12 -> 31
281       | 4 | 6 | 9 | 11           -> 30
282       | 2                       -> if is_leap then 29
283                                   else 28
284       | _ -> raise (Invalid_date "bad month") in
285     if day > max_days then raise (Invalid_date "day too large")
286     else if day < 1 then raise (Invalid_date "day too small")
287     else date ;;
288
289 (* Note the use of "field punning" to simplify the pattern match for
290    the record. By using just the labels in the record, we implicitly
291    define variables for the field values of the same name as the
292    labels: `year`, `month`, and `day`.
293
294    This code also makes use of a useful previously unseen construct of
295    OCaml's matching, which allows the naming of the parts of a data
296    item and the whole item as well. In the pattern `{year; month; day}
297    as date`, the result of the pattern is to name the whole record
298    `date` while still allowing for accessing the parts with names
299    `year`, `month`, and `day`.
300
301    An issue to consider is the priority of the various invariants. It
302    makes sense to check the validity of the year and the month before
303    moving on to check the validity of the day, as we've done here,
304    since the validity of the day relies on the validity of the month
305    and year. That is, in case the year or month is invalid, we'd
306    rather raise an exception complaining about that than one
307    complaining about an invalid day. *)
308
309 (* .....
310    Exercise 9: Define a function `string_of_date` that returns a string
311    representing its date argument. For example,
312
313    # string_of_date {year = 1706; month = 1; day = 17} ;;
314    - : string = "January 17, 1706"
315    .....*)
316
317 (* The trickiest part is converting the month number into a month name
318    string. One method is to index directly into a list of month names,
319    for example, the name for month 5 can be found at index 4 in this
320    0-indexed list:
321
322    # List.nth ["January"; "February"; "March"; "April";

```

```

323         "May"; "June"; "July"; "August";
324         "September"; "October"; "November"; "December"]
325     4 ;;
326     - : string = "May"
327
328     We use this technique below.
329
330     Putting the components together can make use of the string
331     concatenation operator `^`, leading to this implementation:
332
333     let string_of_date ({year; month; day} : date) : string =
334         let month_names = ["January"; "February"; "March"; "April";
335             "May"; "June"; "July"; "August";
336             "September"; "October"; "November"; "December"] in
337         (List.nth month_names (pred month))
338         ^ " "
339         ^ string_of_int day
340         ^ ", "
341         ^ string_of_int year ;;
342
343     Alternatively, functions in the formatted printing module `Printf`
344     are useful to know about for this kind of thing: *)
345
346     let string_of_date ({year; month; day} : date) : string =
347         let month_names = ["January"; "February"; "March"; "April";
348             "May"; "June"; "July"; "August";
349             "September"; "October"; "November"; "December"] in
350         Printf.sprintf "%s %d, %d" (List.nth month_names (pred month)) day year ;;
351
352     (=====
353     Part 3: Persons as an algebraic data type
354
355     Now, combine all of these different types to define a person record,
356     with a name, a favorite color, and a birthdate.
357
358     .....
359     Exercise 10: Define a `person` record type. Use the field names
360     `name`, `favorite`, and `birthdate`.
361     .....*)
362
363     type person = {name : string; favorite : color; birthdate : date} ;;
364
365     (* Just for fun, here's a function to open up a graphics window and
366     display a person's information. Try it out! *)
367
368     (* display_person person -- Displays `person`'s name and birth date in
369     a graphics window on a background of their favorite color. *)
370
371     let display_person ({name; favorite; birthdate} : person) : unit =
372         let open Graphics in
373         open_graph "";
374         resize_window 200 60;
375         let r, g, b = rgb_of_color favorite in
376         set_color (rgb r g b);

```

```

377     fill_rect 0 0 200 60;
378     set_color white;
379     moveto 20 40;
380     draw_string name;
381     moveto 20 20;
382     draw_string (string_of_date birthdate);
383     ignore (read_key ()) ;;
384
385     (*
386     let () = display_person {name = "Ben Franklin";
387                             favorite = Simple Blue;
388                             birthdate = {year = 1706; month = 1; day = 17}} ;;
389     *)

```